Bahan Belajar UTS Mobile Programming (Android, Java)

Mata kuliah di PPT: **Mobile Community Solution**. Bahan ini disusun dari **7 PPT dosen (Sesi 1 sampai 7)** dan **5 sample code**. Hampir semua PPT mengambil materi dari buku *Head First Android Development* (Dawn & David Griffiths), jadi contoh di slide (Messenger, Starbuzz, Band Booking) berasal dari buku itu.

Sesi	Judul PPT	Sample code	Isi
1	Introduction to Android	-	Apa itu Android, versi, arsitektur, struktur project, AVD, APK
2	Multiple Activities and Intent	**02c**	Banyak Activity, manifest, Explicit/Implicit Intent, intent filter, chooser
3	The Activity Lifecycle	**03c**	4 state, 7 method lifecycle, 3 lifetime, Input Controls, Dialog
4	The User Interface	-	RelativeLayout, LinearLayout, GridLayout, padding, margin, gravity, View/ViewGroup
5	RecyclerView and Adapter	**05c**	Komponen RecyclerView, Adapter, ViewHolder, LayoutManager
6	Menu, Themes, Settings	**06a**	4 jenis menu, Style & Theme, Settings/Preferences
7	Data Persistence *(nama file PPT-nya salah, tertulis "Menu, Themes, and Settings")*	**07c**	Shared Preferences, Internal/External Storage, SQLite

> Cakupan UTS: Sesi 1 sampai 7. Materi Services (Sesi 8) tidak dimasukkan.

Cara pakai file ini

- Ada waktu: baca Sesi 1 sampai 7 berurutan.
- Waktu mepet: baca **Rangkuman Kilat** di bagian akhir, lalu kerjakan **Latihan Soal**.
- Tanda `SLIDE` = definisi yang hampir sama persis dengan kalimat di slide. Bagian ini paling mungkin keluar sebagai soal teori.
- Tanda `KODE` = penjelasan kode dosen.

Daftar Isi

SESI 1. Introduction to Android

1.1 Apa itu Android
1.2 Versi Android (dari slide)
1.3 Arsitektur Android (sering keluar)
1.4 Isi sebuah aplikasi Android
1.5 Setting Environment
1.6 Struktur folder project
1.7 Kelas `R` dan `findViewById`
1.8 strings.xml
1.9 Menjalankan aplikasi

SESI 2. Multiple Activities and Intent (project 02c)

2.1 Teori Activity
2.2 Setiap Activity wajib didaftarkan di Manifest
2.3 Intent
2.4 Kode 02c: pindah halaman + kirim data
2.5 Menangani klik
2.6 Mengirim pesan ke aplikasi lain: ACTION_SEND
2.7 Intent Resolution dan Intent Filter
2.8 createChooser
2.9 Layout & widget di 02c

SESI 3. The Activity Lifecycle (project 03c)

3.1 Teori Activity
3.2 Empat state Activity
3.3 Tujuh method lifecycle
3.4 Tiga lifetime
3.5 Bagaimana Activity bekerja
3.6 Kode 03c
3.7 Input Controls
3.8 Dialog

SESI 4. The User Interface

4.1 View dan ViewGroup
4.2 Atribut ukuran
4.3 Padding vs Margin
4.4 RelativeLayout
4.5 LinearLayout
4.6 GridLayout
4.7 ConstraintLayout (dipakai di kode 06a, tidak dibahas di PPT)

SESI 5. RecyclerView and Adapter (project 05c)

5.1 Teori
5.2 Enam komponen RecyclerView
5.3 Layout Manager
5.4 Adapter
5.5 ViewHolder
5.6 Enam langkah implementasi
5.7 Model (POJO) dan klik item

SESI 6. Menu, Themes, Settings (project 06a)

6.1 Empat jenis menu
6.2 App Bar dan Options Menu
6.3 Contextual Menu
6.4 Popup Menu
6.5 Styles and Themes
6.6 Settings

SESI 7. Data Persistence (project 07c)

7.1 Lima pilihan penyimpanan data
7.2 Internal Storage
7.3 External Storage
7.4 Static Resource (res/raw)
7.5 Kenapa Database
7.6 Tiga objek utama SQLite
7.7 SQLiteOpenHelper
7.8 Bagaimana helper memutuskan (onCreate / onUpgrade / onDowngrade)
7.9 Tabel dan tipe data
7.10 Insert, Update, Delete, Query
7.11 Kode 07c: register & login

RANGKUMAN KILAT

Teori (hafalan)
Kode
Error klasik (soal "cari kesalahan")

LATIHAN SOAL (jawaban di bawah)

A. Pilihan ganda
B. Esai singkat
C. Cari kesalahan
D. Lengkapi kode
Kunci Jawaban

SESI 1. Introduction to Android

1.1 Apa itu Android

SLIDE Menurut slide:

- Android adalah **sistem operasi mobile berbasis Linux yang dimodifikasi**.
- Dikembangkan oleh **Android, Inc.**, lalu **dibeli Google tahun 2005**.
- Framework pengembangannya memakai campuran **Java dan XML**.

1.2 Versi Android (dari slide)

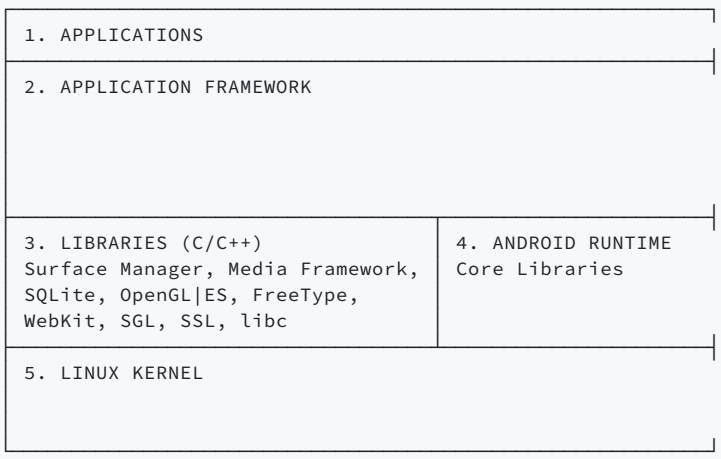
Versi	Nama	Rilis
1.0	-	23 Sep 2008
1.1	-	9 Feb 2009
1.5	Cupcake	30 Apr 2009
1.6	Donut	15 Sep 2009
2.0/2.1	Eclair	26 Okt 2009
2.2	Froyo (Frozen Yogurt)	20 Mei 2010
2.3	Gingerbread	6 Des 2010
3.0	Honeycomb	22 Feb 2011
4.0	Ice Cream Sandwich	19 Okt 2011
4.1	Jelly Bean	9 Jul 2012
4.4	KitKat	31 Okt 2013
5.0	Lollipop	12 Nov 2014
6.0	Marshmallow	5 Okt 2015
7.0	Nougat	22 Agu 2016
8.0	Oreo	21 Agu 2017
9.0	Pie	6 Agu 2018

Polanya: nama makanan manis, urut abjad (Cupcake, Donut, Eclair, Froyo, Gingerbread, Honeycomb, Ice Cream Sandwich, Jelly Bean, KitKat, Lollipop, Marshmallow, Nougat, Oreo, Pie). Pola abjad ini membantu kalau ditanya urutan.

1.3 Arsitektur Android PENTING (sering keluar)

SLIDE "OS Android dibagi menjadi **lima bagian dalam empat layer utama**." Libraries dan Android Runtime berada di layer yang sama, makanya 5 bagian tapi 4 layer.

Dari atas ke bawah:



Bagian	Fungsi (dari keterangan di slide)
Applications	Aplikasi bawaan seperti Contacts, Calendar, Maps, browser. Aplikasi buatan kita juga di layer ini
Application Framework	API yang dipakai aplikasi bawaan, dan kita bisa memakai API yang sama untuk mengatur tampilan dan perilaku aplikasi kita
Libraries	Kumpulan library C dan C++, diakses lewat API framework. Contoh: **SQLite** (database), **WebKit** (browser), **OpenGL ES** (grafis), **SSL** (keamanan)
Android Runtime	Core libraries yang mengimplementasikan sebagian besar bahasa Java. **Setiap aplikasi Android berjalan di prosesnya sendiri**
Linux Kernel	Paling bawah. Menyediakan **driver** hardware, serta layanan inti seperti **keamanan dan manajemen memori**

Contoh soal: "SQLite berada di layer apa?" Jawabannya **Libraries**. "Driver kamera ada di mana?" Jawabannya **Linux Kernel**. "Activity Manager ada di mana?" Jawabannya **Application Framework**.

1.4 Isi sebuah aplikasi Android

Dari gambar di slide, aplikasi terdiri dari tiga hal:

- **Layout**: memberi tahu Android **tampilan** layar.
- **Activity**: menentukan **apa yang dilakukan** aplikasi.
- **Resources**: file tambahan seperti gambar dan suara.

SLIDE Slide 18:

- *Activity* adalah satu hal yang bisa dilakukan user. Biasanya terkait dengan satu layar dan ditulis dalam **Java**. "Activities define actions."
- *Layout* menggambarkan tampilan layar. Ditulis dalam **XML** dan memberi tahu Android cara menyusun elemen di layar. "Layout define how the user interface is presented."

KODE Di semua project dosen, pasangan ini selalu ada: `MainActivity.java` ↔ `activity_main.xml`.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);         // wajib panggil versi induknya
        setContentView(R.layout.activity_main);     // pasang layout ke layar
        // setelah baris ini baru boleh findViewById
    }
}
```

1.5 Setting Environment

- **IntelliJ IDEA** adalah IDE Java populer. **Android Studio adalah versi khusus dari IntelliJ IDEA**.
- Karena berbasis Java, perlu **JDK** (Java Development Kit) dan **JRE** (Java Runtime Environment) yang sesuai.
- Langkah membuat project di Android Studio: **Create New Project → Configure Project → Specify API Level → Create Activity → Configure Activity**.

API Level / Minimum SDK: versi Android paling rendah yang didukung aplikasi. `SLIDE` Kalau kita memilih hanya versi SDK terbaru, aplikasi **tidak bisa berjalan di banyak perangkat** (karena banyak HP masih memakai versi lama). Makin rendah minimum SDK, makin banyak perangkat yang bisa menjalankan, tapi fitur baru tidak bisa dipakai.

1.6 Struktur folder project

`SLIDE` Kategori file menurut slide:

Kategori	Isi
Java and XML Files	File Activity dan layout yang dibuat wizard
Android-generated Java Files	File Java yang dibuat otomatis oleh Android Studio dan tidak perlu disentuh (contohnya kelas `R`)
Resources Files	Gambar ikon, style, dan String yang dipakai aplikasi
Android Libraries	Library Android sesuai minimum SDK yang dipilih
Configuration Files	Memberi tahu Android apa isi aplikasi dan bagaimana menjalankannya (contoh: `AndroidManifest.xml`)

Bentuk foldernya di project dosen:

```
app/src/main/
├── AndroidManifest.xml      ← konfigurasi: daftar Activity, ikon, tema, permission
├── java/com/example/.../    ← kode Java
└── res/                     ← resource
    ├── layout/              ← tampilan (XML)
    ├── menu/                ← isi menu
    ├── values/              ← strings.xml, colors.xml, styles.xml, themes.xml, dimens.xml
    ├── values-night/        ← versi dark mode
    ├── xml/                 ← XML lain (misal pengaturan Preferences)
    ├── raw/                 ← file mentah (Sesi 7)
    └── mipmap/, drawable/   ← ikon & gambar
```

Di luar `app/` ada file **Gradle** (`build.gradle.kts`, `settings.gradle.kts`, `gradlew`). Gradle adalah alat yang meng-compile project dan mengunduh library.

1.7 Kelas `R` dan `findViewById`

Setiap resource otomatis mendapat ID di kelas `R` (termasuk "Android-generated Java files"):

- `R.layout.activity_main` → `res/layout/activity_main.xml`
- `R.id.btnLogin` → komponen dengan `android:id="@+id/btnLogin"`
- `R.string.app_name`, `R.menu.menu_main`, `R.xml.root_preferences`, `R.raw.textfile`

```
<Button android:id="@+id/btnLogin" ... />     <!-- + artinya buat ID baru -->
```

```
Button btnLogin = findViewById(R.id.btnLogin); // ambil SETELAH setContentView
```

`@+id/nama` membuat ID baru. `@id/nama` (tanpa +) merujuk ID yang sudah ada.

1.8 strings.xml

SLIDE Simpan teks di `strings.xml`, **jangan di-hardcode** di layout. `strings.xml` adalah file resource berisi pasangan **nama/nilai** String.

```
<resources>
    <string name="app_name">My Application 02c</string>
    <string name="send">Send Message</string>
</resources>
```

Dipakai di layout: `android:text="@string/send"`. Dipakai di Java: `getString(R.string.send)`. Alasannya: teks mudah diubah di satu tempat dan mudah diterjemahkan ke bahasa lain.

Layout bisa diedit dengan **2 cara**: **code editor** (tulis XML) dan **design editor** (drag-and-drop).

1.9 Menjalankan aplikasi

- **Emulator** menjalankan aplikasi di **AVD (Android Virtual Device)**. AVD berperilaku seperti HP sungguhan. Kita bisa membuat banyak AVD yang meniru perangkat berbeda.
- Langkah membuat AVD: pilih hardware → pilih system image → verifikasi konfigurasi.
- Proses menjalankan: **Compile → Package → Deploy → Run**.
- SLIDE **APK** = *application package*, pada dasarnya file **JAR atau ZIP** untuk aplikasi Android.

SESI 2. Multiple Activities and Intent (project 02c)

2.1 Teori Activity

SLIDE Menurut slide, Activity:

- Komponen aplikasi yang mewakili **satu window** dan satu hierarki view.
- Biasanya memenuhi layar, tapi bisa juga ditanam di Activity lain atau tampil sebagai window mengambang.
- Berupa kelas Java, biasanya satu Activity satu file.
- Menangani interaksi user (klik tombol, input teks, verifikasi login).
- Bisa menjalankan Activity lain di aplikasi yang sama **atau aplikasi lain**.
- Punya siklus hidup (dibahas di Sesi 3).

SLIDE **Task**: kalau beberapa Activity dirangkai untuk melakukan sesuatu yang lebih kompleks, rangkaian itu disebut **task**. Contoh: satu Activity menampilkan daftar resep, Activity lain untuk menambah resep.

2.2 Setiap Activity wajib didaftarkan di Manifest

SLIDE Semua Activity harus dideklarasikan di `AndroidManifest.xml` dengan elemen `<activity>` di dalam `<application>`. Kalau tidak, **sistem tidak tahu Activity itu ada** (aplikasi crash dengan `ActivityNotFoundException`).

```
<application ...>
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".MainActivity2" android:exported="false" />
</application>
```

- `android:name` **wajib**. Diawali titik (`.MainActivity2`) karena Android menggabungkannya dengan nama package menjadi `com.example.myapplication02c.MainActivity2`.
- `android:label` **opsional**, label yang ramah user.
- `MAIN` + `LAUNCHER` = Activity yang dibuka pertama saat ikon aplikasi ditekan.
- `exported="true"` = boleh dibuka aplikasi lain (launcher HP termasuk aplikasi lain).

2.3 Intent

SLIDE **Intent adalah sejenis pesan** ("an intent to do something"). Intent menghubungkan objek terpisah (misalnya Activity) saat runtime. Kalau Activity A ingin menjalankan Activity B, A **mengirim intent ke Android**, lalu Android menjalankan B dan memberikan intent itu kepadanya.

	Explicit Intent	**Implicit Intent**
Menyebut	Nama kelas tujuan	Hanya **aksi** yang diinginkan
Contoh	`new Intent(this, MainActivity2.class)`	`new Intent(Intent.ACTION_VIEW, uri)`, `new Intent(Intent.ACTION_SEND)`
Siapa yang dibuka	Activity yang disebut	Android mencari Activity yang **mampu** menangani aksi itu, bisa dari aplikasi lain

SLIDE Parameter `new Intent(this, Tujuan.class)`: parameter pertama = dari objek mana intent berasal (`this` = Activity sekarang), parameter kedua = kelas Activity yang menerima.

2.4 KODE Kode 02c: pindah halaman + kirim data

Aplikasi: kolom username & password, tombol LOGIN dan Register. LOGIN → pindah ke layar 2 yang menulis "Halo [username]". Register → buka Instagram di browser.

Pengirim (MainActivity):

```
String strUsername = etUsername.getText().toString();   // getText() bukan String, perlu
toString()

Intent niatPindahHalaman = new Intent(this, MainActivity2.class);
niatPindahHalaman.putExtra("keyUsername", strUsername); // titip data (kunci, nilai)
startActivity(niatPindahHalaman);
```

Penerima (MainActivity2):

```
String usernameLama = this.getIntent().getStringExtra("keyUsername");
tvGreetings.setText("Halo " + usernameLama);
```

- SLIDE `putExtra(nama, nilai)` menambah informasi ke intent. Method ini **overloaded**, jadi nilainya bisa banyak tipe (String, int, boolean, dan lain-lain).
- SLIDE `getIntent()` mengembalikan intent yang menjalankan Activity ini, sehingga data tambahannya bisa diambil.
- **Key harus persis sama.** Kalau beda, hasilnya `null` dan layar menampilkan "Halo null".

Buka browser (Implicit Intent):

```
Uri uri = Uri.parse("https://www.instagram.com");
Intent niatBrowser = new Intent(Intent.ACTION_VIEW, uri);
startActivity(niatBrowser);
```

2.5 Menangani klik

Cara dosen (02c): `implements View.OnClickListener`

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {
    protected void onCreate(...) {
        btnLogin.setOnClickListener(this);      // this = Activity ini yang menangani
        btnRegister.setOnClickListener(this);
    }
    @Override
    public void onClick(View view) {
        if (view.getId() == R.id.btnLogin) { ... }
        else if (view.getId() == R.id.btnRegister) { ... }
    }
}
```

Cara di slide (contoh Messenger): atribut `android:onClick` di XML

```
<Button android:text="@string/send" android:onClick="onSendMessage" />
```

```
public void onSendMessage(View view) { ... }   // harus public, void, parameter View
```

Cara ketiga: lambda (dipakai di 05c)

```
btnLogin.setOnClickListener(v -> { /* aksi */ });
```

2.6 Mengirim pesan ke aplikasi lain: ACTION_SEND

Contoh dari slide (aplikasi Messenger):

```
Intent intent = new Intent(Intent.ACTION_SEND);
intent.setType("text/plain");                              // jenis data (MIME type)
intent.putExtra(Intent.EXTRA_TEXT, messageText);           // isi pesan
intent.putExtra(Intent.EXTRA_SUBJECT, "Judul pesan");      // opsional, subjek
startActivity(intent);
```

`setType` dan `EXTRA_TEXT` relevan untuk `ACTION_SEND`, tidak untuk semua aksi.

2.7 Intent Resolution dan Intent Filter PENTING

SLIDE Saat menerima intent, Android harus mencari Activity mana yang bisa menanganinya. Proses ini disebut **intent resolution**.

- Explicit intent: langsung ke kelas yang disebut.
- Implicit intent: Android memeriksa **intent filter** di manifest setiap aplikasi.

```
<activity android:name="ShareActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>          <!-- aksi yang bisa
ditangani -->
        <category android:name="android.intent.category.DEFAULT"/>  <!-- WAJIB untuk implicit
intent -->
        <data android:mimeType="text/plain"/>                       <!-- jenis data yang
diterima -->
        <data android:mimeType="image/*"/>
    </intent-filter>
</activity>
```

SLIDE Intent filter **harus punya category** `DEFAULT`, kalau tidak Activity tidak bisa menerima implicit intent.

Urutan pemeriksaan Android: (1) hanya filter dengan category `DEFAULT`, (2) yang action-nya cocok, (3) yang MIME type-nya cocok.

Hasilnya:

- **Satu** Activity cocok → langsung dibuka.
- **Lebih dari satu** cocok → user diminta memilih, dan bisa memilih "selalu pakai ini".

2.8 createChooser

SLIDE `Intent.createChooser()` menampilkan dialog pemilih:

- Bisa diberi **judul** dialog.
- **Tidak memberi opsi** memilih aplikasi default, jadi user selalu diminta memilih.
- Kalau **tidak ada** Activity yang cocok, tampil pesan pemberitahuan (aplikasi tidak crash).

```
String chooserTitle = getString(R.string.chooser);         // getString = ambil nilai dari
strings.xml
Intent chosenIntent = Intent.createChooser(intent, chooserTitle);
startActivity(chosenIntent);
```

2.9 Layout & widget di 02c

- **LinearLayout vertical** berisi 2 EditText dan LinearLayout horizontal (berisi 2 Button). Ini contoh **nested layout**.
- `android:hint` = teks petunjuk abu-abu.

- `inputType="textPassword"` = huruf disamarkan. `textEmailAddress` = keyboard dengan @.
- Toast: `Toast.makeText(this, "Login ditekan", Toast.LENGTH_SHORT).show();` **Tanpa `.show()` tidak muncul.**
- `etPassword` ada di layout tapi tidak dipakai di Java. Pengecekan password sungguhan baru ada di 07c.
- Untuk menjalankan di HP sungguhan (slide 40-41): aktifkan **USB debugging** di Developer Options.

SESI 3. The Activity Lifecycle (project 03c)

3.1 Teori Activity

SLIDE Menurut slide:

- Activity adalah window yang berisi antarmuka aplikasi.
- Sebuah aplikasi bisa punya **nol atau lebih** Activity.
- Activity di sistem dikelola sebagai **activity stack** (tumpukan). Activity yang baru dibuka ditaruh di atas.
- Sejak muncul sampai tersembunyi, Activity melewati beberapa tahap yang disebut **activity life cycle**.

3.2 Empat state Activity PENTING

State	Kondisi
Active / Running	Di depan layar (di **puncak stack**)
Paused	Kehilangan fokus tapi **masih terlihat** (misalnya tertutup Activity transparan atau tidak penuh). Masih hidup penuh, tapi bisa dimatikan sistem kalau memori **sangat** kurang
Stopped	**Tertutup sepenuhnya** oleh Activity lain. Data masih disimpan tapi tidak terlihat, dan **sering** dimatikan sistem saat memori dibutuhkan
Killed / Destroyed	Activity yang paused/stopped dibuang dari memori (diminta finish atau prosesnya dimatikan). Kalau ditampilkan lagi, harus **dibuat ulang dari awal**

3.3 Tujuh method lifecycle

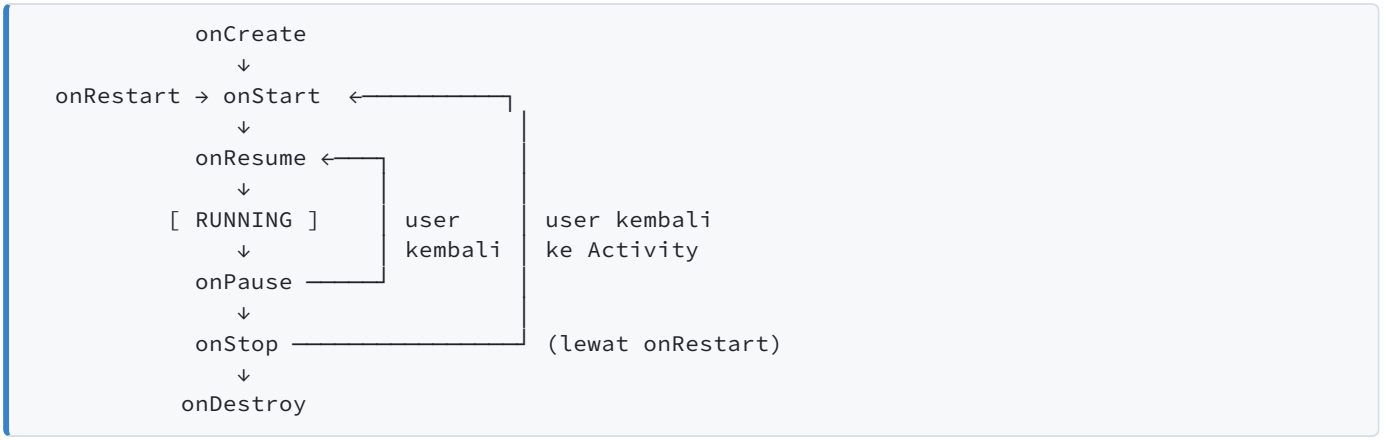
SLIDE Dari slide:

Method	Dipanggil saat
`onCreate()`	Activity **pertama kali dibuat**
`onStart()`	Activity mulai **terlihat** oleh user
`onResume()`	Activity mulai **berinteraksi** dengan user
`onPause()`	Activity sekarang di-pause dan Activity sebelumnya di-resume
`onStop()`	Activity **tidak terlihat** lagi
`onDestroy()`	Sebelum Activity dihancurkan (manual, atau oleh sistem untuk menghemat memori)
`onRestart()`	Activity yang sudah stop **dijalankan lagi**

Pasangannya simetris:

```
onCreate  ↔ onDestroy    (lahir / mati)
onStart   ↔ onStop       (terlihat / tidak terlihat)
onResume  ↔ onPause      (bisa disentuh / tidak)
```

Diagram alurnya:



3.4 Tiga lifetime PENTING

SLIDE Ada tiga "loop" yang bisa dimonitor:

Lifetime	Rentang	Contoh pemakaian (dari slide)
Entire Lifetime	`onCreate()` sampai `onDestroy()`	Siapkan state global di onCreate, lepaskan semua resource di onDestroy. Misalnya thread download dibuat di onCreate dan dihentikan di onDestroy
Visible Lifetime	`onStart()` sampai `onStop()`	User bisa melihat Activity walaupun mungkin tidak di depan. Misalnya daftarkan BroadcastReceiver di onStart, lepas di onStop. **Bisa terjadi berkali-kali**
Foreground Lifetime	`onResume()` sampai `onPause()`	Activity di depan semua Activity lain dan berinteraksi dengan user. Sering berganti antara resumed dan paused (misalnya HP sleep), jadi **kode di sini harus ringan**

3.5 Bagaimana Activity bekerja

SLIDE Dari slide:

- Aplikasi adalah kumpulan Activity, layout, dan resource lain.
- **Secara default setiap aplikasi berjalan di prosesnya sendiri.**
- Activity di aplikasi lain bisa dijalankan dengan intent lewat `startActivity()`.
- Saat Activity perlu dijalankan, Android mengecek apakah sudah ada proses untuk aplikasi itu.
- Saat Android menjalankan Activity, ia memanggil `onCreate()`.

3.6 KODE Kode 03c

Dua Activity. Setiap method lifecycle mencetak log, misalnya `Log.v(TAG, "masuk create 1")`. Klik "Hello World!" → pindah ke MainActivity2. Setiap override wajib memanggil `super.onXxx()`.

Urutan yang muncul di Logcat (sering jadi soal):

Kejadian	Urutan
Buka aplikasi	create 1 → start 1 → resume 1
Pindah ke Activity 2	**pause 1** → create 2 → start 2 → resume 2 → **stop 1**
Back dari Activity 2	pause 2 → (restart 1) → start 1 → resume 1 → stop 2 → **destroy 2**
Tekan Home	pause → stop (tidak destroy)
Kembali dari recent apps	(restart) → start → resume
Rotasi layar	pause → stop → destroy → create → start → resume

Catatan:

- Saat pindah ke Activity 2, Activity 1 **pause dulu**, baru Activity 2 tampil, baru Activity 1 **stop**. Activity 1 **tidak di-destroy**, hanya disimpan di stack.
- `restart 1` tidak tercetak karena dosen tidak meng-override `onRestart`.
- Rotasi menghancurkan dan membuat ulang Activity, sebabnya data yang belum disimpan bisa hilang.

Log vs Toast

- `Log` tampil di **Logcat** (untuk developer). Level: `Log.v` verbose, `Log.d` debug, `Log.i` info, `Log.w` warning, `Log.e` error. TAG = label untuk filter.
- `Toast` tampil ke **user**.

Layout 03c: RelativeLayout (teori lengkapnya di Sesi 4)

```
[Hello World!][ login ]
[ register ]
               [Saklar Lampu o]
```

Yang diberi listener adalah `tvHelloWorld` (TextView), bukan Button. Artinya **View apa pun bisa diklik**.

3.7 Input Controls PENTING

SLIDE Basic views dari slide:

Control	Fungsi	Catatan dari slide
Button	Teks dan/atau ikon yang menunjukkan aksi saat disentuh	
Text Field (EditText)	User mengetik teks, bisa satu baris atau banyak baris	Menyentuhnya menaruh kursor dan memunculkan keyboard
Checkbox	Memilih **beberapa** opsi dari satu set	Jangan pakai satu checkbox untuk on/off, **pakai Switch**
Radio Button	Memilih **satu** opsi dari satu set	Pakai kalau user perlu melihat semua opsi berdampingan. Kalau tidak, pakai **Spinner** yang lebih hemat tempat
Toggle Button	Mengubah setting antara **dua state**	Android 4.0 (API 14) memperkenalkan **Switch** dengan kontrol geser
Spinner	Memilih satu nilai dari daftar **dropdown**	Default menampilkan nilai yang sedang dipilih
Picker	Memilih **tanggal atau waktu** lewat dialog siap pakai	Menjamin format valid dan sesuai locale user

Checkbox vs Radio Button: **checkbox = boleh banyak**, **radio = hanya satu** (radio dibungkus `RadioGroup`).

3.8 Dialog PENTING

SLIDE **Dialog** = window kecil yang meminta user mengambil keputusan atau memasukkan informasi. **Tidak memenuhi layar**, biasanya untuk kejadian **modal** (user harus bertindak dulu sebelum lanjut).

SLIDE Kelas `Dialog` adalah kelas dasar, tapi **jangan dibuat langsung**. Pakai subclass:

- **AlertDialog**: bisa berisi judul, sampai 3 tombol, daftar pilihan, atau layout custom.
- **DatePickerDialog / TimePickerDialog**: UI siap pakai untuk memilih tanggal/waktu.

SLIDE **Tiga area AlertDialog:**

1. **Title**: opsional, dipakai hanya kalau area konten berisi pesan detail, daftar, atau layout custom.

2. **Content area**: pesan, daftar, atau layout custom.
3. **Action buttons**: **tidak boleh lebih dari tiga**.

Contoh kode AlertDialog (tidak ada di kode dosen, tapi pola standarnya begini):

```
new AlertDialog.Builder(this)
    .setTitle("Logout")
    .setMessage("Yakin mau keluar?")
    .setPositiveButton("Ya", (dialog, which) -> { /* aksi */ })
    .setNegativeButton("Batal", null)
    .show();
```

SLIDE **Popup**: versi ringan dialog, user memilih satu hal. **Tidak punya tombol OK/Cancel**. Memilih item langsung melanjutkan alur, dan menyentuh di luar popup menutupnya.

SLIDE **Toast**: umpan balik ringan dalam popup kecil, **hilang otomatis** setelah beberapa saat. Contoh: keluar dari email sebelum dikirim memunculkan toast "Draft saved".

SESI 4. The User Interface

4.1 View dan ViewGroup PENTING

SLIDE Dari slide:

- UI terdiri dari **layout** dan **komponen GUI**.
- **Komponen GUI adalah sejenis View**, yaitu objek yang menempati ruang di layar (bentuknya persegi panjang).
- **Layout adalah sejenis View yang disebut ViewGroup**, yaitu View khusus yang **bisa berisi View lain**.
- **View hierarchy**: saat aplikasi di-build, XML layout diubah menjadi objek **ViewGroup yang berisi pohon View**.

```
LinearLayout (ViewGroup)
├── EditText (View)
├── EditText (View)
└── LinearLayout (ViewGroup)
    ├── Button (View)
    └── Button (View)
```

SLIDE Keuntungan menjadi View:

1. **Getting and setting properties**: setiap View adalah objek Java, jadi propertinya bisa diambil/diubah dari kode Activity.
2. **Size and position**: lebar dan tinggi bisa ditentukan.
3. **Focus handling**: Android mengatur perpindahan fokus.
4. **Event handling and listeners**: View bisa merespons event, dan kita bisa memasang listener.

4.2 Atribut ukuran

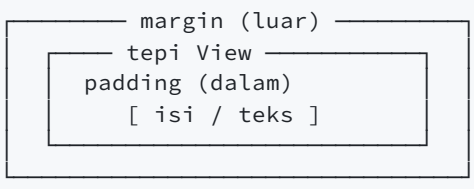
SLIDE `layout_width` dan `layout_height` **wajib** untuk semua layout dan View.

Nilai	Arti
`wrap_content`	Cukup besar untuk menampung isinya
`match_parent`	Sebesar parent-nya (misalnya selebar layar dikurangi padding)
`fill_parent`	Nama lama, **sudah diganti** `match_parent`

Satuan: `dp` untuk ukuran umum (menyesuaikan kerapatan layar), `sp` khusus ukuran teks (ikut pengaturan font user).

4.3 Padding vs Margin PENTING

- **Padding** = jarak **di dalam** View/layout, antara tepinya dan isinya. Atribut: `android:padding`, `paddingTop`, `paddingBottom`, `paddingLeft`, `paddingRight`.
- **Margin** = jarak **di luar** View, antara View itu dan View lain. Atribut: `layout_marginTop`, `layout_marginBottom`, `layout_marginLeft`, `layout_marginRight`.



SLIDE Nilai padding sebaiknya disimpan di **dimension resource file** `res/values/dimens.xml` supaya padding di semua layout mudah dirawat. Android membaca nilainya saat runtime.

```
<!-- dimens.xml -->
<resources>
    <dimen name="activity_horizontal_margin">16dp</dimen>
</resources>
<!-- layout -->
android:paddingLeft="@dimen/activity_horizontal_margin"
```

4.4 RelativeLayout

Posisi View ditentukan **relatif terhadap parent** atau **relatif terhadap View lain**.

SLIDE Supaya View selalu di posisi tertentu apa pun ukuran/orientasi layar, posisikan **relatif terhadap parent**:

Atribut	Fungsi
`layout_alignParentTop` / `Bottom` / `Left` / `Right`	Tepi View disejajarkan dengan tepi parent
`layout_centerInParent`	Di tengah parent (horizontal dan vertikal)
`layout_centerHorizontal`	Di tengah secara horizontal
`layout_centerVertical`	Di tengah secara vertikal

SLIDE Untuk memposisikan **relatif terhadap View lain**, View acuan (anchor) **harus punya ID**:

Atribut	Fungsi
`layout_above`	Di atas View acuan
`layout_below`	Di bawah View acuan
`layout_toLeftOf`	Di kiri View acuan
`layout_toRightOf`	Di kanan View acuan
`layout_alignTop` / `alignBottom`	Tepi atas/bawah sejajar dengan View acuan
`layout_alignLeft` / `alignRight`	Tepi kiri/kanan sejajar dengan View acuan

KODE Di 03c:

```
<Button android:id="@+id/btnLogin"    android:layout_toRightOf="@id/tvHelloWorld" />
<Button android:id="@+id/btnRegister" android:layout_below="@id/tvHelloWorld" />
<Switch android:layout_toRightOf="@id/btnLogin" android:layout_below="@id/btnRegister" />
```

4.5 LinearLayout

SLIDE Menyusun View **dalam urutan sesuai urutan di XML**. Arah ditentukan `android:orientation`: `vertical` (atas ke bawah) atau `horizontal` (kiri ke kanan).

Weight PENTING SLIDE `android:layout_weight` membagi **sisa ruang** (remaining space) di layout. Contoh dari slide: field "To" weight 1, field "Message" weight 2. Total 1+2 = 3. "To" dapat **1/3** sisa ruang, "Message" dapat **2/3**.

KODE Di 02c, dua tombol masing-masing `layout_weight="1"` dengan `weightSum="2"`, jadi sisa ruang dibagi rata. Karena lebarnya `wrap_content`, yang dibagi adalah ruang **setelah** lebar teks masing-masing tombol. Supaya persis setengah-setengah, lebar tombol diisi `0dp`.

gravity vs layout_gravity PENTING (sering tertukar)

	`android:gravity`	`android:layout_gravity`
Mengatur	Posisi **isi** (misalnya teks) **di dalam** View	Posisi **View itu sendiri** di dalam **wadahnya**
Contoh	Teks di dalam EditText ditaruh di atas: `gravity="top"`	Tombol digeser ke kanan LinearLayout: `layout_gravity="end"`

Nilai `gravity`: `top`, `bottom`, `left`, `right`, `center_vertical`, `center_horizontal`, `center`, `fill_vertical`, `fill_horizontal`, `fill`. Nilai `layout_gravity`: sama, ditambah `start` dan `end` (awal/akhir wadah).

4.6 GridLayout

`SLIDE` Membagi layar menjadi **baris dan kolom**, lalu menempatkan View di sel-selnya.

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:columnCount="2">                <!-- jumlah kolom -->

    <TextView android:layout_row="0" android:layout_column="0" ... />
    <EditText android:layout_row="0" android:layout_column="1" ... />

    <Button
        android:layout_row="1"
        android:layout_column="0"
        android:layout_columnSpan="2" />     <!-- melebar 2 kolom -->
</GridLayout>
```

- `columnCount` = jumlah kolom. `rowCount` = jumlah baris maksimum.
- `layout_row` dan `layout_column` = posisi View. **Dimulai dari 0.**
- `layout_columnSpan` = View melebar ke beberapa kolom.
- Saran slide: **mulai dari sketsa**, tentukan butuh berapa kolom.

4.7 ConstraintLayout (dipakai di kode 06a, tidak dibahas di PPT)

```
<TextView
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

View diikat ke tepi parent. Diikat ke atas dan bawah sekaligus, juga kiri dan kanan sekaligus, hasilnya **tepat di tengah layar**.

Perbandingan cepat:

Layout	Cara menyusun
LinearLayout	Berderet satu arah sesuai urutan XML
RelativeLayout	Relatif terhadap parent atau View lain
GridLayout	Baris dan kolom
ConstraintLayout	Diikat (constraint) ke parent atau View lain

SESI 5. RecyclerView and Adapter (project 05c)

5.1 Teori

SLIDE Dari slide:

- RecyclerView adalah **wadah yang bisa di-scroll untuk data dalam jumlah besar**.
- **Efisien**: memakai dan memakai ulang (reuse) View dalam jumlah terbatas, dan cepat memperbarui data yang berubah.
- Lebih fleksibel daripada **ListView**, tapi **butuh lebih banyak setup**.
- Mengelola sedikit View untuk memberi kesan koleksi besar yang melebihi layar.

Kenapa hemat: kalau ada 100 data, RecyclerView hanya membuat View sebanyak yang terlihat (misalnya 10 baris). Baris yang keluar layar **didaur ulang** untuk data berikutnya.

5.2 Enam komponen RecyclerView PENTING

SLIDE Dari slide:

Komponen	Kelas / file	Di 05c
Data	Sumber data	`ArrayList<Song>`
RecyclerView	List yang bisa di-scroll	`rvSongs` di `activity_main.xml`
Layout satu item	File XML	`row_song.xml`
Layout manager	`RecyclerView.LayoutManager`	`LinearLayoutManager`
Adapter	`RecyclerView.Adapter`	`SongAdapter`
ViewHolder	`RecyclerView.ViewHolder`	`SongViewHolder`

SLIDE Hubungannya:

- RecyclerView mengakses datanya **lewat adapter**.
- **Adapter bukan bawaan Android.** Kita menulis adapter sendiri yang disesuaikan dengan data: menentukan tipe data, membuat view, dan mengisi (bind) data ke view.
- Item diposisikan oleh **layout manager**.

5.3 Layout Manager

SLIDE Dari slide:

- Setiap ViewGroup punya layout manager.
- Dipakai untuk memposisikan item di RecyclerView.
- **Memakai ulang item yang tidak lagi terlihat oleh user.**
- Tiga bawaan:
  - `LinearLayoutManager`: list (vertikal atau horizontal)
  - `GridLayoutManager`: grid
  - `StaggeredGridLayoutManager`: grid dengan tinggi item berbeda-beda
- Bisa juga membuat sendiri dengan extend `RecyclerView.LayoutManager`.

5.4 Adapter PENTING

SLIDE Dari slide:

- **Membantu interface yang tidak cocok untuk bekerja sama.** Contoh: mengambil data dari Cursor database lalu menyiapkannya sebagai String untuk View.

- **Perantara antara data dan View.**
- Mengelola pembuatan, pembaruan, penambahan, dan penghapusan item View saat data berubah.

SLIDE **Adapter punya 3 method wajib:** `onCreateViewHolder()`, `onBindViewHolder()`, `getItemCount()`. (Di slide tertulis "inBindViewHolder", itu salah ketik. Yang benar `onBindViewHolder`.)

KODE `SongAdapter.java`:

```
public class SongAdapter extends RecyclerView.Adapter<SongAdapter.SongViewHolder> {

    ArrayList<Song> songsInAdapter;

    // (1) Membuat ViewHolder dari row_song.xml
    @Override
    public SongViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.row_song, parent, false);
        return new SongViewHolder(view);
    }

    // (2) Mengisi data posisi ke-sekian ke ViewHolder
    @Override
    public void onBindViewHolder(SongViewHolder holder, int position) {
        Song song = songsInAdapter.get(position);
        holder.tvSongTitle.setText(song.getTitle());
        holder.tvSongArtist.setText(song.getArtist());
    }

    // (3) Jumlah data
    @Override
    public int getItemCount() {
        return songsInAdapter.size();
    }
}
```

Analogi: `onCreateViewHolder` = mencetak formulir kosong, `onBindViewHolder` = mengisi formulir dengan data orang ke-sekian, `getItemCount` = memberi tahu ada berapa orang.

`LayoutInflater.inflate` mengubah file XML menjadi objek View. Parameter `false` = jangan langsung ditempel ke parent, karena RecyclerView yang menempelkannya.

5.5 ViewHolder

SLIDE Dari slide: dipakai adapter untuk **menyiapkan satu View berisi data untuk satu item list**. Layout-nya ditentukan di file XML, bisa punya elemen yang bisa diklik, dan diposisikan oleh layout manager.

```
public class SongViewHolder extends RecyclerView.ViewHolder {
    TextView tvSongTitle, tvSongArtist;
    public SongViewHolder(View itemView) {               // constructor ViewHolder
        super(itemView);
        tvSongTitle = itemView.findViewById(R.id.tvSongTitle);   // dari itemView, bukan
Activity
        tvSongArtist = itemView.findViewById(R.id.tvSongArtist);
    }
}
```

ViewHolder **menyimpan referensi View** satu baris, sehingga `findViewById` tidak diulang setiap scroll.

5.6 Enam langkah implementasi PENTING

SLIDE Dari slide:

1. Tambahkan dependency RecyclerView di `build.gradle` (kalau perlu).
2. Tambahkan RecyclerView ke layout.
3. Buat layout XML untuk satu item.
4. Extend `RecyclerView.Adapter`.
5. Extend `RecyclerView.ViewHolder`.
6. Di `onCreate()` Activity, buat RecyclerView dengan adapter dan layout manager.

KODE Langkah 6 di 05c:

```
songs = new ArrayList<>();
for (int i = 0; i < 100; i++) {
    songs.add(new Song("Judul " + i, "Artis " + i));
}
rvSongs = findViewById(R.id.rvSongs);
rvSongs.setLayoutManager(new LinearLayoutManager(this));   // WAJIB, kalau lupa list kosong
rvSongs.setAdapter(songAdapter);
```

Ganti layout manager: `new GridLayoutManager(this, 2)` untuk grid 2 kolom.

5.7 KODE Model (POJO) dan klik item

`Song.java` = **POJO** (Plain Old Java Object), kelas biasa berisi atribut, constructor, getter, setter.

Klik item memakai **interface** sebagai jembatan, karena Adapter tidak tahu harus berbuat apa, yang memutuskan adalah Activity:

```
// Di Adapter
public interface OnItemClickListener { void onItemClick(Song song); }

holder.itemView.setOnClickListener(v -> {
    if (listener != null) listener.onItemClick(song);
});
```

```
// Di MainActivity, diisi dengan lambda
SongAdapter songAdapter = new SongAdapter(songs, clickedSong ->
    Toast.makeText(this, clickedSong.getTitle() + " is clicked", Toast.LENGTH_LONG).show());
```

Lambda `clickedSong -> {...}` adalah singkatan dari membuat objek `new SongAdapter.OnItemClickListener() { public void onItemClick(Song clickedSong) {...} }`.

Tinggi root `row_song.xml` harus `wrap_content`. Kalau `match_parent`, satu baris memenuhi satu layar.

SESI 6. Menu, Themes, Settings (project 06a)

6.1 Empat jenis menu PENTING

SLIDE Dari slide:

1. **App bar dengan options menu**
2. **Context menu** (floating context menu)
3. **Contextual action bar** (contextual action mode)
4. **Popup menu**

6.2 App Bar dan Options Menu

SLIDE **App bar** = bar di atas setiap layar. Isinya:

1. Ikon navigasi untuk membuka navigation drawer
2. Judul Activity saat ini
3. Ikon untuk item options menu
4. **Action overflow button** (titik tiga) untuk sisa options menu

SLIDE **Options menu**: item penting tampil sebagai ikon di app bar, sisanya muncul saat titik tiga ditekan. Letaknya di **pojok kanan** app bar. Dipakai untuk **pindah ke Activity lain dan mengedit pengaturan aplikasi**.

SLIDE **Tiga langkah membuat options menu:**

1. **Buat resource menu**: folder `res/menu`, file `menu_main.xml`, tambahkan item.
2. **Override** `onCreateOptionsMenu()` untuk meng-inflate menu.
3. **Override** `onOptionsItemSelected()` untuk menangani klik.

Ikon menu bisa ditambahkan lewat: klik kanan `drawable` → **New → Image Asset** → pilih **Action Bar and Tab Items** → pilih clipart → Next → Finish.

KODE `menu_main.xml` di 06a:

```
<item android:id="@+id/menu_help"     android:title="Help"     app:showAsAction="always" />
<item android:id="@+id/menu_settings" android:title="Settings" app:showAsAction="always" />
<item android:id="@+id/menu_logout"   android:title="Log Out"  app:showAsAction="ifRoom" />
```

`showAsAction`	Arti
`always`	Selalu tampil di app bar
`ifRoom`	Tampil di app bar kalau muat, kalau tidak masuk overflow (titik tiga)
`never`	Selalu di overflow (titik tiga)

KODE `MainActivity.java` di 06a:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;                                    // true = tampilkan menu
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.menu_help) {
        Toast.makeText(this, "help di klik", Toast.LENGTH_SHORT).show();
        return true;                                // true = klik sudah ditangani
    } else if (id == R.id.menu_settings) {
        startActivity(new Intent(this, SettingsActivity.class));
        return true;
    } else if (id == R.id.menu_logout) {
        ...
        return true;
    }
    return super.onOptionsItemSelected(item);       // selain itu serahkan ke induk
}
```

6.3 Contextual Menu

SLIDE Dari slide:

- Memungkinkan user melakukan aksi pada **View yang dipilih**.
- Bisa dipasang di View apa saja, tapi paling sering untuk item di RecyclerView, GridView, atau koleksi View lain.
- **Dua jenis:**

	Floating context menu	**Contextual action mode**
Muncul saat	**Long press** pada View	Long press, muncul **bar sementara** menggantikan/di bawah app bar
Jumlah View	Aksi pada **satu View** sekaligus	Aksi pada **banyak View** sekaligus

Langkah floating context menu:

1. Buat resource `menu_context.xml`.
2. Daftarkan View di `onCreate`: `registerForContextMenu(view);`
3. Override `onCreateContextMenu` untuk meng-inflate menu:

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenu.ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu_context, menu);
}
```

4. Tangani klik di `onContextItemSelected(MenuItem item)`.

Langkah contextual action bar:

1. Buat resource menu dan beri ikon.
2. Pasang `setOnLongClickListener()` di View, lalu panggil `startActionMode()`.
3. Implementasi `mActionModeCallback` (`ActionMode.Callback`) dengan 4 method:
    - `onCreateActionMode`: inflate menu

- `onPrepareActionMode`: dipanggil setiap action mode ditampilkan, **selalu setelah** `onCreateActionMode`, bisa berkali-kali
- `onActionItemClicked`: menangani klik aksi
- `onDestroyActionMode`: saat action mode ditutup

Bar tetap tampil sampai user menekan **Done** (panah kiri). Contoh aksinya Edit, Share, Delete.

6.4 Popup Menu

SLIDE Dari slide:

- Daftar item vertikal yang **ditambatkan (anchored) ke sebuah View**, biasanya ke ikon.
- Aksinya **tidak langsung memengaruhi isi View**. Contoh: di aplikasi email, Reply All dan Forward berkaitan dengan email tapi tidak mengubah email itu.

Langkah:

1. Buat resource menu XML.
2. Tambahkan `ImageButton` untuk ikon popup di layout.
3. Pasang `onClickListener` ke tombol.
4. Implementasi `onClick`: buat dan tampilkan popup.
5. Implementasi `onMenuItemClick`: tangani klik item.

```
PopupMenu popup = new PopupMenu(this, view);
popup.getMenuInflater().inflate(R.menu.menu_popup, popup.getMenu());
popup.setOnMenuItemClickListener(item -> { /* tangani klik */ return true; });
popup.show();
```

Ringkasan method tiap menu:

Menu	Inflate	Tangani klik
Options menu	`onCreateOptionsMenu`	`onOptionsItemSelected`
Floating context menu	`onCreateContextMenu`	`onContextItemSelected`
Contextual action mode	`onCreateActionMode`	`onActionItemClicked`
Popup menu	`popup.getMenuInflater().inflate(...)`	`onMenuItemClick`

6.5 Styles and Themes PENTING

SLIDE **Style** = kumpulan properti yang menentukan tampilan dan format sebuah **View atau window** (tinggi, padding, warna font, ukuran font, warna background, dan lain-lain). Didefinisikan di resource XML **terpisah** dari XML layout. Filosofinya mirip **CSS** di web: memisahkan desain dari konten.

SLIDE **Theme** = style yang diterapkan ke **seluruh Activity atau aplikasi**, bukan ke satu View. Semua View di dalamnya akan memakai properti style itu yang didukungnya.

Pewarisan (inheritance) style, dua cara:

- Mewarisi style **bawaan Android** → pakai atribut `parent`:

  ```
  <style name="MyJumboText" parent="Widget.AppCompat.TextView"> ... </style>
  ```

- Mewarisi style **buatan sendiri** → SLIDE tidak perlu `parent`, cukup **awali nama style baru dengan nama style induk, dipisah titik**:

  ```
  <style name="MyJumboText.blue"> ... </style>
  ```

KODE `styles.xml` di 06a memakai **kedua cara**:

```
<style name="MyJumboText" parent="Widget.AppCompat.TextView">      <!-- cara 1 -->
    <item name="android:textColor">@color/red</item>
    <item name="android:textSize">48sp</item>
    <item name="android:textStyle">bold</item>
</style>

<style name="MyJumboText.blue">                                    <!-- cara 2 -->
    <item name="android:textColor">@color/blue</item>
</style>
```

Dipakai di layout: `style="@style/MyJumboText.blue"`. Hasilnya **biru, 48sp, bold**: mewarisi semua atribut `MyJumboText`, lalu **menimpa** warna merah menjadi biru.

Warna dari `colors.xml`: `@color/blue` = `#FF0000FF`. Formatnya `#AARRGGBB` (AA = transparansi, FF = tidak transparan).

Menerapkan theme (SLIDE dari slide):

- Untuk **seluruh aplikasi**: atribut `android:theme` di tag `<application>` di manifest.

  ```
  <application android:theme="@style/CustomTheme">
  ```

- Untuk **satu Activity**: atribut `android:theme` di tag `<activity>`.
- Ada theme bawaan, misalnya background transparan: `@android:style/Theme.Translucent`.
- Untuk memodifikasi theme bawaan, jadikan theme itu `parent` dari theme custom.

KODE Di 06a:

- `values/themes.xml` = theme mode terang, `values-night/themes.xml` = theme mode gelap (dipakai otomatis saat dark mode).
- **Temuan menarik:** theme dark mode memakai `NoActionBar`, jadi **di dark mode action bar dan menu Help/Settings/Log Out hilang**. `SettingsActivity` juga diberi theme `NoActionBar` di manifest, sehingga `getSupportActionBar()` mengembalikan `null` dan panah back tidak tampil. Karena itu kode memakai `if (actionBar != null)` supaya tidak crash.

6.6 Settings PENTING

SLIDE Dari slide:

- Settings memungkinkan user mengatur fitur dan perilaku aplikasi. Contoh: lokasi rumah, satuan ukuran default, perilaku notifikasi.
- Untuk nilai yang **jarang berubah** dan relevan bagi sebagian besar user.
- Kalau nilainya **sering berubah**, pakai **options menu** atau **navigation drawer**.
- User mengakses settings lewat **navigation drawer** atau **options menu**.
- Taruh di bawah label **"Settings"**.

SLIDE **Aturan jumlah pengaturan:**

Jumlah	Cara menyusun
7 atau kurang	Urutkan berdasarkan prioritas, yang paling penting di atas
7 sampai 15	Kelompokkan pengaturan terkait dengan **section divider** (PreferenceCategory)
16 atau lebih	Kelompokkan ke **layar terpisah** (subscreen) yang dibuka dari layar settings utama

SLIDE Settings didefinisikan di **preference screen**, mirip layout, di `res/xml/preferences.xml` (di 06a namanya `root_preferences.xml`).

Atribut Preference:

- SLIDE `key`: **wajib** untuk preference yang menyimpan nilai. Ini kunci unik yang dipakai sistem saat menyimpan nilai ke **SharedPreferences**. Tidak wajib hanya untuk `PreferenceCategory`, `PreferenceScreen`, atau preference yang membuka Intent/Fragment.
- SLIDE `title`: nama setting yang dilihat user.
- SLIDE `defaultValue`: nilai awal di SharedPreferences. **Sebaiknya diberikan untuk semua setting.**
- `dependency`: preference ini hanya aktif kalau preference lain (yang key-nya disebut) aktif.
- `summary`: keterangan di bawah judul.

Mengelompokkan:

- Dengan judul/divider → bungkus dengan `<PreferenceCategory android:title="...">`.
- Dengan subscreen → bungkus dengan `<PreferenceScreen>` di dalam PreferenceScreen.

Jenis preference: `CheckBoxPreference` (di slide), `SwitchPreferenceCompat`, `EditTextPreference`, `ListPreference` (di kode dosen), `RingtonePreference` (di slide).

KODE `root_preferences.xml` di 06a:

Elemen	Bentuk	Key
`PreferenceCategory` "Messages"	Judul kelompok	-
`EditTextPreference`	Isian teks "Your signature"	`signature`
`ListPreference`	Pilih "Reply" / "Reply to all"	`reply`
`PreferenceCategory` "Sync"	Judul kelompok	-
`SwitchPreferenceCompat`	Saklar "Sync email periodically"	`sync`
`SwitchPreferenceCompat` + `dependency="sync"`	Saklar yang **hanya aktif kalau sync menyala**	`attachment`

- `entries` = teks yang dilihat user ("Reply to all"), `entryValues` = nilai yang disimpan ("reply_all"). Keduanya dari `arrays.xml`.
- `useSimpleSummaryProvider="true"` = nilai yang dipilih tampil di bawah judul.
- `summaryOn` / `summaryOff` = keterangan berbeda saat saklar on/off.
- Slide memakai `android:` (Preference versi lama), kode dosen memakai `app:` (AndroidX). Konsepnya sama.

Implementasi: Settings Activity + Settings Fragment SLIDE Template **Basic Activity** sudah menyertakan options menu dengan item **Settings**.

KODE `SettingsActivity.java`:

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.settings_activity);      // isinya FrameLayout id "settings"
    if (savedInstanceState == null) {
        getSupportFragmentManager()
            .beginTransaction()
            .replace(R.id.settings, new SettingsFragment())
            .commit();
    }
    ActionBar actionBar = getSupportActionBar();
    if (actionBar != null) actionBar.setDisplayHomeAsUpEnabled(true);  // panah back
}

public static class SettingsFragment extends PreferenceFragmentCompat {
    @Override
    public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {
        setPreferencesFromResource(R.xml.root_preferences, rootKey);
    }
}
```

- **Fragment** = potongan UI yang ditempel di dalam Activity. **FrameLayout** jadi wadahnya.
- FragmentTransaction: `beginTransaction()` → `replace(wadah, fragment)` → `commit()`. Tanpa `commit()` tidak terjadi apa-apa.
- `if (savedInstanceState == null)` mencegah Fragment ditempel dua kali saat Activity dibuat ulang (misalnya rotasi), karena Fragment lama dipulihkan otomatis.
- `PreferenceFragmentCompat` membangun layar settings dari XML dan **menyimpan pilihan user otomatis** ke SharedPreferences.

Membaca nilainya:

```
SharedPreferences prefs = PreferenceManager.getDefaultSharedPreferences(this);
boolean sync = prefs.getBoolean("sync", false);
String signature = prefs.getString("signature", "");
```

SESI 7. Data Persistence (project 07c)

> Nama file PPT-nya "Sesi 7 Menu, Themes, and Settings", tapi slide judulnya **"Data Persistence"**.

7.1 Lima pilihan penyimpanan data PENTING

SLIDE Pilihan tergantung kebutuhan: apakah data **privat** untuk aplikasi atau bisa diakses aplikasi lain/user, dan berapa besar ruang yang dibutuhkan.

Pilihan	Fungsi
Shared Preferences	Data **primitif privat** dalam pasangan **key-value**
Internal Storage	Data **privat** di memori perangkat
External Storage	Data **publik** di penyimpanan eksternal bersama (misalnya SD card)
SQLite Database	Data **terstruktur** di database privat
Network Connection	Data di web, di server sendiri

SLIDE **SharedPreferences** menyimpan data sebagai pasangan nama/nilai, misalnya user ID, tanggal lahir, gender, nomor SIM. Untuk data seperti teks puisi, lebih cocok memakai file system dengan kelas dari package `java.io`.

7.2 Internal Storage PENTING

SLIDE Dari slide:

- File disimpan langsung di penyimpanan internal perangkat.
- Secara default **privat**: aplikasi lain (dan user) **tidak bisa mengaksesnya**.
- Saat aplikasi **di-uninstall, file ikut terhapus**.

Menulis (3 langkah):

1. Panggil `openFileOutput()` dengan nama file dan mode → menghasilkan `FileOutputStream`.
2. Tulis dengan `write()`.
3. Tutup dengan `close()`.

Membaca (3 langkah):

1. Panggil `openFileInput()` dengan nama file → menghasilkan `FileInputStream`.
2. Baca dengan `read()`.
3. Tutup dengan `close()`.

```
// Tulis
FileOutputStream fos = openFileOutput("textfile.txt", MODE_PRIVATE);
fos.write(str.getBytes());
fos.close();

// Baca
FileInputStream fis = openFileInput("textfile.txt");
// ... baca dengan fis.read(...)
fis.close();
```

Mode file:

Mode	Arti
`MODE_PRIVATE`	SLIDE Membuat file (atau **mengganti** file bernama sama) dan menjadikannya privat
`MODE_APPEND`	Menambah di akhir file yang sudah ada
`MODE_WORLD_READABLE` / `MODE_WORLD_WRITEABLE`	Bisa dibaca/ditulis aplikasi lain

> Untuk UTS jawab sesuai slide. Di Android modern, `MODE_WORLD_READABLE` dan `MODE_WORLD_WRITEABLE` sudah tidak boleh dipakai karena alasan keamanan.

Contoh aplikasi di slide: ketik teks → tekan **Save** → Toast "File saved successfully!" dan teks hilang dari EditText → tekan **Load** → teks muncul lagi. Itu bukti file tersimpan (`textfile.txt`).

7.3 External Storage

SLIDE Dari slide:

- Berguna karena **kapasitas lebih besar** dan file **mudah dibagikan** (misalnya SD card dipindah ke orang lain).
- Path didapat dari `getExternalStorageDirectory()`. Biasanya `/sdcard` di HP dan `/mnt/sdcard` di emulator.
- **Jangan hardcode path SD card**, karena tiap produsen bisa memakai nama path berbeda.
- Contoh slide membuat folder `MyFiles` di SD card lalu menyimpan file di sana.
- SLIDE Butuh **permission** di manifest:

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

> Sama seperti di atas: `getExternalStorageDirectory()` sudah usang di Android baru, tapi untuk UTS ikuti slide.

7.4 Static Resource (res/raw)

SLIDE Dari slide:

- File bisa ditambahkan ke package saat **design time** untuk dipakai saat runtime. Contoh: file bantuan (help).
- Taruh di folder **`res/raw`** (folder ini dibuat sendiri).
- Dibuka dengan `getResources().openRawResource(R.raw.textfile)` → menghasilkan `InputStream`.
- **Resource ID = nama file tanpa ekstensi**: `textfile.txt` → `R.raw.textfile`.
- File di `res/raw` **hanya bisa dibaca**, tidak bisa ditulis.

7.5 Kenapa Database

SLIDE Dari slide:

- Teknik sebelumnya cocok untuk data sederhana. Untuk **data relasional**, database jauh lebih efisien.
- Contoh: nilai ujian semua siswa sekolah. Dengan database kita bisa query nilai siswa tertentu.
- Database bisa menjaga **integritas data** lewat relasi antar data.
- Android memakai **SQLite**.
- Database aplikasi **hanya bisa diakses aplikasi itu sendiri**.

7.6 Tiga objek utama SQLite PENTING

SLIDE Dari slide:

Objek	Fungsi	Analogi JDBC
SQLite Helper (extend `SQLiteOpenHelper`)	Membuat dan mengelola database	-
SQLiteDatabase	Memberi akses ke database	Seperti `SQLConnection`
Cursor	Membaca dari dan menulis ke database (menelusuri hasil query)	Seperti `ResultSet`

SLIDE Pakai `getWritableDatabase()` untuk **menulis** dan `getReadableDatabase()` untuk **membaca**. Keduanya mengembalikan objek `SQLiteDatabase`.

SLIDE Setiap query mengembalikan **Cursor** yang menunjuk ke semua baris hasil query. Cursor adalah mekanisme untuk menelusuri hasil query dan membaca baris serta kolom. Untuk query kompleks (misalnya butuh alias kolom) pakai `SQLiteQueryBuilder`.

7.7 SQLiteOpenHelper

SLIDE Dua informasi yang dibutuhkan helper untuk membuat database: **nama database** dan **versi database**.

- `onCreate()` dipanggil saat database **pertama kali dibuat** di perangkat.
- `onUpgrade()` dipanggil saat database **perlu di-upgrade**.

KODE `MySQLiteOpenHelper.java`:

```
public class MySQLiteOpenHelper extends SQLiteOpenHelper {
    static final String DB_NAME = "my_db";
    static final int DB_VERSION = 1;

    public MySQLiteOpenHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(UserDAO.CREATE_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) { }
}
```

`onCreate` milik helper **berbeda** dengan `onCreate` milik Activity. Namanya sama, tugasnya lain.

7.8 Bagaimana helper memutuskan (onCreate / onUpgrade / onDowngrade) PENTING

SLIDE Dari diagram slide:

Kondisi	Yang terjadi
1. Database **belum ada**	Helper membuat database, lalu `onCreate()` dipanggil
2. Database **sudah ada**	Helper membandingkan versi di database dengan versi di helper
3. Versi helper **lebih besar**	`onUpgrade()` dipanggil, lalu versi database diperbarui

Kondisi	Yang terjadi
4. Versi helper **lebih kecil**	`onDowngrade()` dipanggil, lalu versi database diperbarui
5. Versi **sama**	Tidak ada method yang dipanggil, database sudah up to date

SLIDE `onUpgrade()` punya **3 parameter**: database SQLite, nomor versi database saat ini (lama), dan versi baru yang diberikan ke superclass.

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion == 1) {
        db.execSQL("ALTER TABLE DRINK ADD COLUMN FAVORITE NUMERIC;");
    }
}
```

SLIDE `onDowngrade()` dipakai untuk mengembalikan database ke versi sebelumnya.

AWAS **Jebakan di 07c:** kalau `CREATE_TABLE` diubah tanpa menaikkan `DB_VERSION`, tabel tidak berubah karena `onCreate` tidak dipanggil lagi (kondisi 5). Solusinya naikkan versi dan isi `onUpgrade`, atau uninstall aplikasi.

7.9 Tabel dan tipe data

SLIDE Data di SQLite disimpan dalam **tabel**. Tabel punya beberapa **baris**, setiap baris dibagi menjadi **kolom**, dan setiap kolom berisi satu data.

SLIDE **Lima storage class / tipe data SQLite:** `INTEGER`, `TEXT`, `REAL`, `NUMERIC`, `BLOB`.

SLIDE **Konvensi Android: kolom primary key dinamai `_id`.** Kode Android mengharapkan ada kolom `_id`, dan mengabaikannya membuat data lebih sulit ditampilkan di UI.

> Perhatikan: kode dosen di 07c memakai `id`, bukan `_id`. Ini bisa jadi soal "apa yang tidak sesuai konvensi".

```
CREATE TABLE DRINK (_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    NAME TEXT,
                    DESCRIPTION TEXT,
                    IMAGE_RESOURCE_ID INTEGER)
```

KODE Versi 07c:

```
"CREATE TABLE IF NOT EXISTS ms_user ("
+ "id INTEGER PRIMARY KEY AUTOINCREMENT, "   // id terisi otomatis 1, 2, 3...
+ "username TEXT UNIQUE, "                   // username tidak boleh kembar
+ "password TEXT )"
```

Nama tabel dan kolom disimpan di konstanta `static final` supaya tidak salah ketik di banyak tempat.

7.10 Insert, Update, Delete, Query PENTING

Insert SLIDE Data dimasukkan sebagai pasangan nama/nilai ke **`ContentValues`** dengan `put()`, lalu `db.insert(...)`.

```
ContentValues drinkValues = new ContentValues();
drinkValues.put("NAME", "Latte");
drinkValues.put("DESCRIPTION", "Espresso and steamed milk");
db.insert("DRINK", null, drinkValues);
// bentuk umum: db.insert(String table, String nullColumnHack, ContentValues values)
```

Hasilnya **id baris baru**, atau **-1 kalau gagal**.

Update SLIDE Tentukan tabel, nilai baru, dan kondisinya.

```
ContentValues drinkValues = new ContentValues();
drinkValues.put("DESCRIPTION", "Tasty");
db.update("DRINK", drinkValues, "NAME = ?", new String[] {"Latte"});
// = UPDATE DRINK SET DESCRIPTION = 'Tasty' WHERE NAME = 'Latte'
```

Kondisi banyak:

```
db.update("DRINK", drinkValues,
          "NAME = ? OR DESCRIPTION = ?",
          new String[] {"Latte", "Our best drip coffee"});
```

Kondisi angka (harus diubah jadi String):

```
db.update("DRINK", drinkValues, "_id = ?", new String[] {Integer.toString(1)});
```

Delete (SLIDE mirip update, tanpa ContentValues):

```
db.delete("DRINK", "NAME = ?", new String[] {"Latte"});
```

Mengubah struktur tabel dengan SQL (dijalankan dengan `db.execSQL(...)`):

```
ALTER TABLE DRINK ADD COLUMN FAVORITE NUMERIC   -- tambah kolom
ALTER TABLE DRINK RENAME TO FOO                 -- ganti nama tabel
DROP TABLE DRINK                                -- hapus tabel
```

7.11 KODE Kode 07c: register & login

Aplikasi: kolom username & password, tombol login dan register, teks "users:".

- **Register** → simpan ke database.
- **Login** → cek kombinasi username & password. Toast menampilkan `true` / `false`.
- **Klik "users:"** → tampilkan semua user (`1. budi`, `2. ani`, ...).
- Data **tetap ada** walaupun aplikasi ditutup, berbeda dengan ArrayList di 05c.

Arsitektur:

```
MainActivity ──► UserDAO ──► MySQLiteOpenHelper ──► database "my_db"
 (tampilan)    (query SQL)    (buat/buka DB)
                  └── menghasilkan objek User (model)
```

DAO (Data Access Object) = kelas yang mengumpulkan semua operasi database untuk satu tabel. Activity tidak perlu menulis SQL.

registerUser (Create)

```
ContentValues cv = new ContentValues();
cv.put("username", username);
cv.put("password", password);
SQLiteDatabase db = mySQLiteOpenHelper.getWritableDatabase();
long result = db.insert("ms_user", null, cv);
db.close();
return result;              // -1 kalau gagal, misalnya username kembar (UNIQUE)
```

getAllUsers (Read semua)

```
SQLiteDatabase db = mySQLiteOpenHelper.getReadableDatabase();
Cursor cursor = db.query("ms_user", null, null, null, null, null, null);   // SELECT * FROM
ms_user

if (cursor != null && cursor.moveToFirst()) {        // ke baris pertama; false kalau kosong
    int indexId = cursor.getColumnIndex("id");       // cari nomor kolom dari namanya
    int indexUsername = cursor.getColumnIndex("username");
    int indexPassword = cursor.getColumnIndex("password");
    do {
        users.add(new User(cursor.getInt(indexId),
                           cursor.getString(indexUsername),
                           cursor.getString(indexPassword)));
    } while (cursor.moveToNext());                   // ke baris berikut; false kalau habis
    cursor.close();
    db.close();
}
```

Pakai `do...while` karena `moveToFirst()` sudah menaruh Cursor di baris pertama, jadi baris itu harus diproses sebelum `moveToNext()`.

getSpecificUser (Read untuk login)

```
String[] columns = { "id" };
String selection = "username = ? AND password = ?";
String[] selectionArgs = { username, password };
Cursor cursor = db.query("ms_user", columns, selection, selectionArgs, null, null, null);
// = SELECT id FROM ms_user WHERE username = ? AND password = ?
int cursorCount = cursor.getCount();
cursor.close();
return cursorCount > 0;
```

Urutan parameter `db.query`:

```
query(table, columns, selection, selectionArgs, groupBy, having, orderBy)
       FROM   SELECT   WHERE     nilai untuk ?  GROUP BY HAVING  ORDER BY
```

`columns = null` artinya semua kolom.

Kenapa pakai `?`: nilai dari `selectionArgs` dimasukkan dengan aman sebagai data, bukan sebagai perintah SQL. Ini mencegah **SQL Injection** (user mengetik `' OR '1'='1` untuk membobol login).

OOP: constructor dan NullPointerException `User.java` punya 3 constructor (**overloading**: nama sama, parameter beda). Komentar dosen:

```
userA = new User();            // objek dibuat di RAM, username = ""
Log.v("asd", userA.username);  // aman
```

Kalau `new User()` dihapus, `userA` bernilai `null`, dan `userA.username` membuat **crash NullPointerException**. Prinsip yang sama berlaku untuk `findViewById` sebelum `setContentView`.

Kelemahan kode 07c (bisa jadi soal analisis):

1. Password disimpan apa adanya (plain text), seharusnya di-hash.
2. Hasil `registerUser` tidak dicek, jadi user tidak tahu kalau gagal.
3. Di `getAllUsers`, `cursor.close()` dan `db.close()` hanya di dalam `if`. Kalau tabel kosong, keduanya tidak ditutup.
4. Di `getSpecificUser`, `db.close()` tidak dipanggil.
5. Input kosong tetap bisa di-register.
6. Primary key dinamai `id`, bukan `_id` sesuai konvensi Android di slide.

RANGKUMAN KILAT

Teori (hafalan)

Topik	Inti
Android	OS mobile berbasis Linux modifikasi, Android Inc., dibeli Google 2005
Arsitektur	5 bagian, 4 layer: Applications, Application Framework, Libraries + Android Runtime, Linux Kernel
APK	Application package, seperti JAR/ZIP
AVD	Android Virtual Device, dijalankan di emulator
Android Studio	Versi khusus IntelliJ IDEA
Activity vs Layout	Activity (Java) = apa yang dilakukan. Layout (XML) = tampilan
Task	Rangkaian beberapa Activity
Intent	Sejenis pesan, "niat melakukan sesuatu"
Intent resolution	Proses Android mencari Activity yang bisa menangani intent
Intent filter	Wajib category `DEFAULT` untuk menerima implicit intent
createChooser	Selalu tampil pemilih, ada judul, tidak ada opsi default, aman kalau tidak ada aplikasi cocok
4 state	Active/Running, Paused, Stopped, Killed
3 lifetime	Entire (Create-Destroy), Visible (Start-Stop), Foreground (Resume-Pause)
Checkbox vs Radio	Checkbox banyak pilihan, Radio satu pilihan
AlertDialog	3 area: title, content, action buttons (maksimal 3)
Popup	Tanpa tombol, sentuh di luar untuk menutup
View vs ViewGroup	GUI component = View. Layout = ViewGroup (berisi View lain)
Padding vs Margin	Padding di dalam, margin di luar
gravity vs layout_gravity	gravity = posisi isi di dalam View. layout_gravity = posisi View di wadahnya
layout_weight	Membagi sisa ruang secara proporsional
RecyclerView	6 komponen: Data, RecyclerView, item layout, LayoutManager, Adapter, ViewHolder
Adapter	Perantara data dan View, bukan bawaan Android, 3 method wajib
3 LayoutManager	Linear, Grid, StaggeredGrid
4 jenis menu	Options menu, Context menu, Contextual action bar, Popup menu
Style vs Theme	Style untuk View. Theme untuk seluruh Activity/aplikasi
Inheritance style	Bawaan pakai `parent`. Buatan sendiri pakai prefix titik
Aturan settings	≤7 urut prioritas, 7-15 pakai divider, ≥16 subscreen
Preference `key`	Wajib, kunci penyimpanan di SharedPreferences
5 penyimpanan	SharedPreferences, Internal, External, SQLite, Network
Internal storage	Privat, terhapus saat uninstall, `openFileOutput` / `openFileInput`
External storage	Publik, butuh `WRITE_EXTERNAL_STORAGE`, jangan hardcode path

Topik	Inti
res/raw	`openRawResource(R.raw.namafile)`, hanya baca
3 objek SQLite	Helper, SQLiteDatabase (≈SQLConnection), Cursor (≈ResultSet)
Tipe data SQLite	INTEGER, TEXT, REAL, NUMERIC, BLOB
Konvensi PK	`_id`
Versi DB	Belum ada → onCreate. Helper > DB → onUpgrade. Helper < DB → onDowngrade. Sama → tidak ada

Kode

Topik	Inti
Activity	`onCreate` → `super.onCreate` → `setContentView` → `findViewById`
Explicit Intent	`new Intent(this, Tujuan.class)` + `startActivity`
Kirim data	`putExtra("key", nilai)` ↔ `getIntent().getStringExtra("key")`
Implicit Intent	`new Intent(Intent.ACTION_VIEW, Uri.parse(url))`
ACTION_SEND	`setType("text/plain")` + `putExtra(Intent.EXTRA_TEXT, ...)`
Toast	`Toast.makeText(this, "pesan", Toast.LENGTH_SHORT).show()`
A ke B	pause A → create B → start B → resume B → stop A
Back dari B	pause B → restart A → start A → resume A → stop B → destroy B
Rotasi	pause → stop → destroy → create → start → resume
Adapter	`onCreateViewHolder`, `onBindViewHolder`, `getItemCount`
RecyclerView	`setLayoutManager(...)` + `setAdapter(...)`
Options menu	`onCreateOptionsMenu` (inflate) + `onOptionsItemSelected` (klik)
Context menu	`registerForContextMenu` + `onCreateContextMenu` + `onContextItemSelected`
Fragment	`beginTransaction().replace(wadah, fragment).commit()`
Insert	`ContentValues` + `getWritableDatabase().insert(tabel, null, cv)`
Update	`db.update(tabel, cv, "kolom = ?", new String[]{nilai})`
Delete	`db.delete(tabel, "kolom = ?", new String[]{nilai})`
Query	`getReadableDatabase().query(...)` → Cursor → `moveToFirst` / `moveToNext`

Error klasik (soal "cari kesalahan")

1. `findViewById` sebelum `setContentView` → NullPointerException.
2. Toast tanpa `.show()` → tidak muncul.
3. Key `putExtra` ≠ key `getStringExtra` → `null`.
4. Activity tidak didaftarkan di manifest → `ActivityNotFoundException`.
5. Intent filter tanpa category `DEFAULT` → tidak menerima implicit intent.
6. RecyclerView tanpa `setLayoutManager` → list kosong.
7. Lupa `super.onCreate(...)` → crash (`SuperNotCalledException`).
8. FragmentTransaction tanpa `commit()` → fragment tidak tampil.
9. Ubah struktur tabel tanpa menaikkan `DB_VERSION` → tabel tidak berubah.

10. Objek dipakai tanpa `new` → NullPointerException.
11. Tulis ke external storage tanpa permission → gagal.
12. Item `row_song.xml` tingginya `match_parent` → satu item memenuhi satu layar.

LATIHAN SOAL (jawaban di bawah)

A. Pilihan ganda

1. SQLite dalam arsitektur Android berada di layer...
   a. Linux Kernel b. Libraries c. Application Framework d. Applications
2. Driver kamera dan WiFi berada di layer...
   a. Linux Kernel b. Android Runtime c. Libraries d. Application Framework
3. APK pada dasarnya adalah file...
   a. EXE b. JAR/ZIP c. XML d. DEX saja
4. Agar Activity bisa menerima implicit intent, intent filter harus memiliki category...
   a. LAUNCHER b. MAIN c. DEFAULT d. BROWSABLE
5. Method lifecycle saat Activity mulai bisa berinteraksi dengan user adalah...
   a. onCreate b. onStart c. onResume d. onRestart
6. Activity yang kehilangan fokus tapi masih terlihat berada di state...
   a. Running b. Paused c. Stopped d. Destroyed
7. Visible lifetime berlangsung antara...
   a. onCreate dan onDestroy b. onStart dan onStop c. onResume dan onPause d. onRestart dan onStop
8. Control yang tepat untuk memilih **beberapa** opsi sekaligus adalah...
   a. Radio Button b. Spinner c. Checkbox d. Toggle Button
9. Jumlah maksimal action button di AlertDialog adalah...
   a. 1 b. 2 c. 3 d. 4
10. Atribut untuk mengatur posisi **teks di dalam** sebuah EditText adalah...
    a. layout_gravity b. gravity c. layout_margin d. padding
11. Field "To" weight 1 dan "Message" weight 3. Bagian sisa ruang untuk "Message" adalah...
    a. 1/3 b. 1/4 c. 3/4 d. 2/3
12. Atribut GridLayout agar sebuah View melebar ke 2 kolom adalah...
    a. columnCount="2" b. layout_column="2" c. layout_columnSpan="2" d. layout_weight="2"
13. Berikut adalah LayoutManager bawaan RecyclerView, **kecuali**...
    a. LinearLayoutManager b. GridLayoutManager c. StaggeredGridLayoutManager d. RelativeLayoutManager
14. Method Adapter yang mengisi data ke tiap baris adalah...
    a. onCreateViewHolder b. onBindViewHolder c. getItemCount d. onCreate
15. `app:showAsAction="ifRoom"` artinya...
    a. Selalu di app bar b. Selalu di overflow c. Di app bar kalau muat d. Hanya di landscape
16. Aplikasi punya 12 pengaturan. Menurut slide, sebaiknya...
    a. Diurutkan berdasarkan prioritas saja b. Dikelompokkan dengan section divider c. Dipisah ke subscreen d. Dimasukkan ke options menu
17. Penyimpanan untuk data primitif privat dalam bentuk key-value adalah...
    a. SQLite b. External Storage c. Shared Preferences d. Network
18. File di internal storage akan...
    a. Bisa diakses semua aplikasi b. Terhapus saat aplikasi di-uninstall c. Pindah ke SD card d. Butuh permission WRITE_EXTERNAL_STORAGE
19. Dalam analogi JDBC, `Cursor` mirip dengan...
    a. SQLConnection b. Statement c. ResultSet d. DriverManager
20. Versi di SQLite helper = 3, versi database di HP = 2. Method yang dipanggil...
    a. onCreate b. onUpgrade c. onDowngrade d. Tidak ada

21. `db.insert(...)` mengembalikan -1 artinya...
    a. Berhasil di baris 1 b. Gagal c. Tabel kosong d. Database tertutup

B. Esai singkat

22. Sebutkan 5 bagian arsitektur Android dan jelaskan satu fungsi masing-masing.
23. Jelaskan beda Explicit Intent dan Implicit Intent, beserta contoh kodenya.
24. Tulis urutan method lifecycle saat user pindah dari MainActivity ke MainActivity2, lalu menekan Back.
25. Sebutkan dan jelaskan 3 lifetime Activity.
26. Apa beda padding dan margin? Apa beda gravity dan layout_gravity?
27. Sebutkan 6 komponen RecyclerView dan 6 langkah implementasinya.
28. Sebutkan 4 jenis menu di Android dan beda floating context menu dengan contextual action mode.
29. Apa beda style dan theme? Jelaskan dua cara pewarisan style.
30. Style `MyJumboText.blue` dipakai di TextView. Sebutkan warna, ukuran, dan ketebalan teksnya beserta alasannya.
31. Sebutkan 5 pilihan penyimpanan data di Android.
32. Jelaskan 5 kondisi yang diperiksa SQLiteOpenHelper terkait versi database.
33. Kenapa query login memakai `"username = ? AND password = ?"`?

C. Cari kesalahan

34.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Button btn = findViewById(R.id.btnLogin);
    btn.setOnClickListener(this);
    setContentView(R.layout.activity_main);
}
```

35.

```
// MainActivity
intent.putExtra("keyUsername", "budi");
// MainActivity2
String nama = getIntent().getStringExtra("username");
```

36.

```
<activity android:name=".ShareActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>
</activity>
<!-- ShareActivity tidak pernah muncul di pilihan aplikasi saat share. Kenapa? -->
```

37.

```
rvSongs = findViewById(R.id.rvSongs);
rvSongs.setAdapter(songAdapter);
// list tidak muncul
```

38. Developer menambah kolom `email` di `CREATE_TABLE`, lalu menjalankan ulang aplikasi tanpa uninstall. Insert email gagal karena kolom tidak ditemukan. Kenapa, dan bagaimana memperbaikinya?

D. Lengkapi kode

39. Membuka `https://www.google.com`:

```
Uri uri = ________("https://www.google.com");
Intent niat = new Intent(________, uri);
________(niat);
```

40. Mengubah DESCRIPTION menjadi "Tasty" untuk minuman bernama "Latte":

```
ContentValues cv = new ContentValues();
cv.________("DESCRIPTION", "Tasty");
db.________("DRINK", cv, "NAME = ?", new String[] {"Latte"});
```

41. Menampilkan options menu:

```
@Override
public boolean ________(Menu menu) {
    getMenuInflater().________(R.menu.menu_main, menu);
    return true;
}
```

Kunci Jawaban

A. Pilihan ganda

1. b
2. a
3. b
4. c
5. c
6. b
7. b
8. c
9. c
10. b
11. c (3 dari total 1+3 = 4)
12. c
13. d
14. b
15. c
16. b
17. c
18. b
19. c
20. b
21. b

B. Esai

22. Applications: aplikasi bawaan dan aplikasi kita. Application Framework: API untuk membangun aplikasi (Activity Manager, dan lain-lain). Libraries: library C/C++ seperti SQLite dan WebKit. Android Runtime: core libraries Java, setiap aplikasi di proses sendiri. Linux Kernel: driver hardware, keamanan, manajemen memori.

23. Explicit menyebut kelas tujuan: `new Intent(this, MainActivity2.class)`. Implicit hanya menyebut aksi, lalu Android mencari Activity yang cocok lewat intent filter: `new Intent(Intent.ACTION_VIEW, Uri.parse(url))`.
24. Pindah: pause 1 → create 2 → start 2 → resume 2 → stop 1. Back: pause 2 → restart 1 → start 1 → resume 1 → stop 2 → destroy 2.
25. Entire lifetime (onCreate sampai onDestroy, menyiapkan dan melepas resource global). Visible lifetime (onStart sampai onStop, Activity terlihat, bisa berkali-kali). Foreground lifetime (onResume sampai onPause, Activity di depan dan berinteraksi, kodenya harus ringan).
26. Padding = jarak di dalam View antara tepi dan isinya. Margin = jarak di luar View ke View lain. Gravity = posisi isi di dalam View. layout_gravity = posisi View di dalam wadahnya.
27. Komponen: Data, RecyclerView, layout satu item, LayoutManager, Adapter, ViewHolder. Langkah: tambah dependency, tambah RecyclerView ke layout, buat layout item, extend Adapter, extend ViewHolder, pasang adapter dan layout manager di onCreate.
28. Options menu, context menu, contextual action bar, popup menu. Floating context menu muncul saat long press dan untuk satu View. Contextual action mode menampilkan bar sementara di tempat app bar dan bisa untuk banyak View sekaligus.
29. Style untuk satu View, theme untuk seluruh Activity/aplikasi (di manifest). Mewarisi style bawaan Android pakai atribut `parent`. Mewarisi style sendiri cukup dengan prefix nama + titik (`MyJumboText.blue`).
30. Biru, 48sp, bold. `MyJumboText.blue` mewarisi `MyJumboText` (merah, 48sp, bold) lewat prefix titik, lalu menimpa warnanya menjadi biru.
31. Shared Preferences, Internal Storage, External Storage, SQLite Database, Network Connection.
32. (1) DB belum ada → buat DB dan onCreate. (2) DB ada → bandingkan versi. (3) Versi helper lebih besar → onUpgrade. (4) Versi helper lebih kecil → onDowngrade. (5) Sama → tidak ada method dipanggil.
33. Mencegah SQL Injection. Nilai dari `selectionArgs` dimasukkan sebagai data, bukan sebagai perintah SQL.

C. Cari kesalahan

34. `findViewById` dipanggil sebelum `setContentView`, jadi `btn` bernilai `null` dan crash NullPointerException. Pindahkan `setContentView` tepat setelah `super.onCreate`.
35. Key tidak sama (`"keyUsername"` vs `"username"`), jadi `nama` bernilai `null`.
36. Intent filter tidak punya `<category android:name="android.intent.category.DEFAULT"/>`, sehingga tidak bisa menerima implicit intent.
37. Belum memasang LayoutManager. Tambahkan `rvSongs.setLayoutManager(new LinearLayoutManager(this));`.
38. Database sudah ada dan versinya sama, jadi `onCreate` tidak dipanggil lagi dan tabel lama tidak berubah. Perbaikannya: naikkan `DB_VERSION` dan tambahkan `ALTER TABLE ms_user ADD COLUMN email TEXT` di `onUpgrade`, atau uninstall aplikasi.

D. Lengkapi kode

39. `Uri.parse`, `Intent.ACTION_VIEW`, `startActivity`.
40. `put`, `update`.
41. `onCreateOptionsMenu`, `inflate`.

Semangat UTS-nya!